

NextWatch

System Architecture, Tech Stack, & Free Hosting Execution Plan

Prepared for: **Anti Gravity**

1. System Architecture

The system is designed as a decoupled client-server architecture. The frontend handles state and UI themes, while the backend acts as a secure API Gateway to process logic and proxy external requests without exposing API keys.

Data Flow & Architecture

1. **Client (Browser):** User selects Anime or Movie, answers the questionnaire, and hits "Find My NextWatch".
2. **Frontend (React/Next.js):** Collects inputs into a JSON payload and sends a POST request to the custom Backend API.
3. **Backend API Gateway (Python/Node):**
 - Receives payload.
 - If "Anime" -> Constructs request to Jikan API / AniList GraphQL.
 - If "Movie/TV" -> Constructs request to TMDB API (appending JustWatch provider data).
 - Aggregates ratings (IMDb/MAL/AL) and formats a unified response.
4. **External APIs:** Process the request and return raw data to the Backend.
5. **Backend to Frontend:** Backend cleans up the raw data and sends a standardized JSON object back to the Client.
6. **Client (Browser):** Renders the results as premium digital poster cards.

2. Technical Stack

Layer	Technology	Why for this MVP?
Frontend	React.js (Next.js), Tailwind CSS, Framer Motion	Next.js provides fast initial loads. Tailwind enables rapid styling and easy light/dark mode toggling. Framer Motion handles smooth transitions.
Backend API	Python (FastAPI) or Node (Express)	FastAPI is extremely fast and auto-generates documentation (Swagger). Express is great if the team strictly prefers JavaScript. Both are lightweight API gateways.
Database	Supabase (PostgreSQL)	While the MVP mainly fetches external data, a database is needed to store user accounts (optional), saved watchlists, and cached API responses to save bandwidth.

3. 100% Free Hosting Strategy (2026 Standards)

To run the MVP with zero server costs, we will leverage the best free tiers available in 2026:

• **Frontend Hosting: Vercel or Cloudflare Pages**

Vercel is the dominant platform for Next.js, offering instant CDN, unlimited deployments, and a generous free tier for non-commercial use. Cloudflare Pages is an excellent alternative with no bandwidth limits.

• **Backend Hosting: PandaStack or Render**

PandaStack provides a highly comprehensive free tier that includes container deployments and managed databases, making it an excellent all-in-one choice for 2026. Alternatively, **Render** allows free web service hosting directly from Git, though it spins down after 15 minutes of inactivity (causing cold starts).

• **Database & Auth: Supabase**

Supabase offers a massive free tier including 500MB database storage, generous bandwidth, and built-in user authentication.

4. Execution Plan (4-Phase MVP Rollout)

Phase 1: Prototyping & Setup (Week 1)

- Initialize GitHub repository.
- Design the UI wireframes in Figma (Unified landing page, Anime vs Movie paths, Results cards).
- Obtain API keys for TMDB and Jikan.

Phase 2: Backend API Gateway Development (Week 2)

- Initialize Python (FastAPI) or Node (Express) project.
- Create the two core endpoints: `/api/recommend/anime` and `/api/recommend/movie`.
- Write the logic to fetch and normalize data from TMDB and Jikan.
- Deploy backend to **PandaStack** or **Render** to get the live API URL.

Phase 3: Frontend Development (Week 3)

- Scaffold the Next.js app with Tailwind.
- Build the interactive Questionnaire components and state management (Zustand).
- Integrate frontend with the live Backend API URL.
- Build the Results Gallery UI (Posters, Match Scores, OTT logos).

Phase 4: Deployment & Polish (Week 4)

- Deploy the Next.js frontend to **Vercel**.
- Set up Supabase for user "Watchlists" (Optional MVP+ feature).
- Test the end-to-end flow for bugs and mobile responsiveness.
- Launch the MVP.